

DAY 18



CHAINS & AGENTS

Chains: Extending the Capabilities of LLMs



- LangChain is named after one of its main methods, chains. Although we can run LLMs in isolation, their power is shown when used with additional components or even when used in conjunction with each other. Chains not only allow for extending the capabilities of LLMs but also for multiple chains to be connected together.
- The most basic form of a chain in LangChain is a single chain. Although a chain can take many forms, each with a different complexity, it generally connects an LLM with some additional tool, prompt, or feature. This idea of connecting a component to an LLM is illustrated in Figure 7-3.

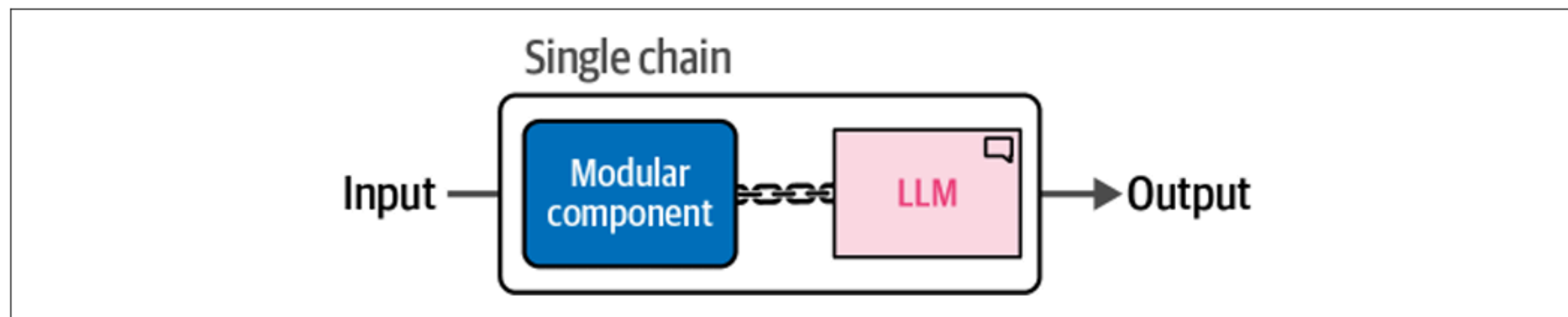


Figure 7-3. A single chain connects some modular component, like a prompt template or external memory, to the LLM.

A Single Link in the Chain: Prompt Template



- We start with creating our first chain, namely the prompt template that Phi-3 expects. In the previous chapter, we explored how transformers.pipeline applies the chat template automatically. This is not always the case with other packages and they might need the prompt template to be explicitly defined. With LangChain, we will use chains to create and use a default prompt template. It also serves as a nice hands-on experience with using prompt templates.
- The idea, as illustrated in Figure 7-4, is that we chain the prompt template together with the LLM to get the output we are looking for. Instead of having to copy-paste the prompt template each time we use the LLM, we would only need to define the user and system prompts.

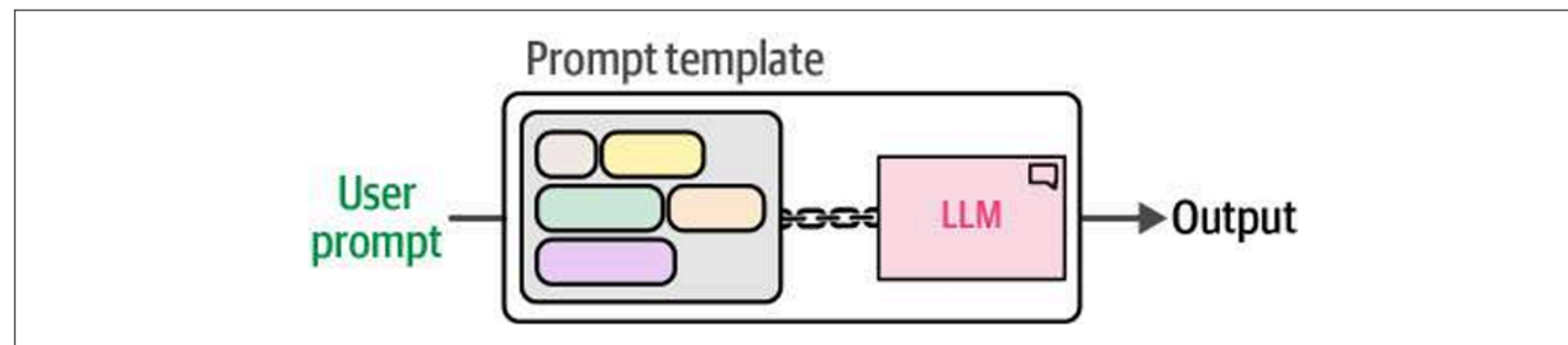


Figure 7-4. By chaining a prompt template with an LLM, we only need to define the input prompts. The template will be constructed for you.

- The template for Phi-3 is comprised of four main components:
- to indicate when the prompt starts
- `<|user|>` to indicate the start of the user's prompt
- `<|assistant|>` to indicate the start of the model's output
- `<|end|>` to indicate the end of either the prompt or the model's output

These are further illustrated in Figure 7-5 with an example.

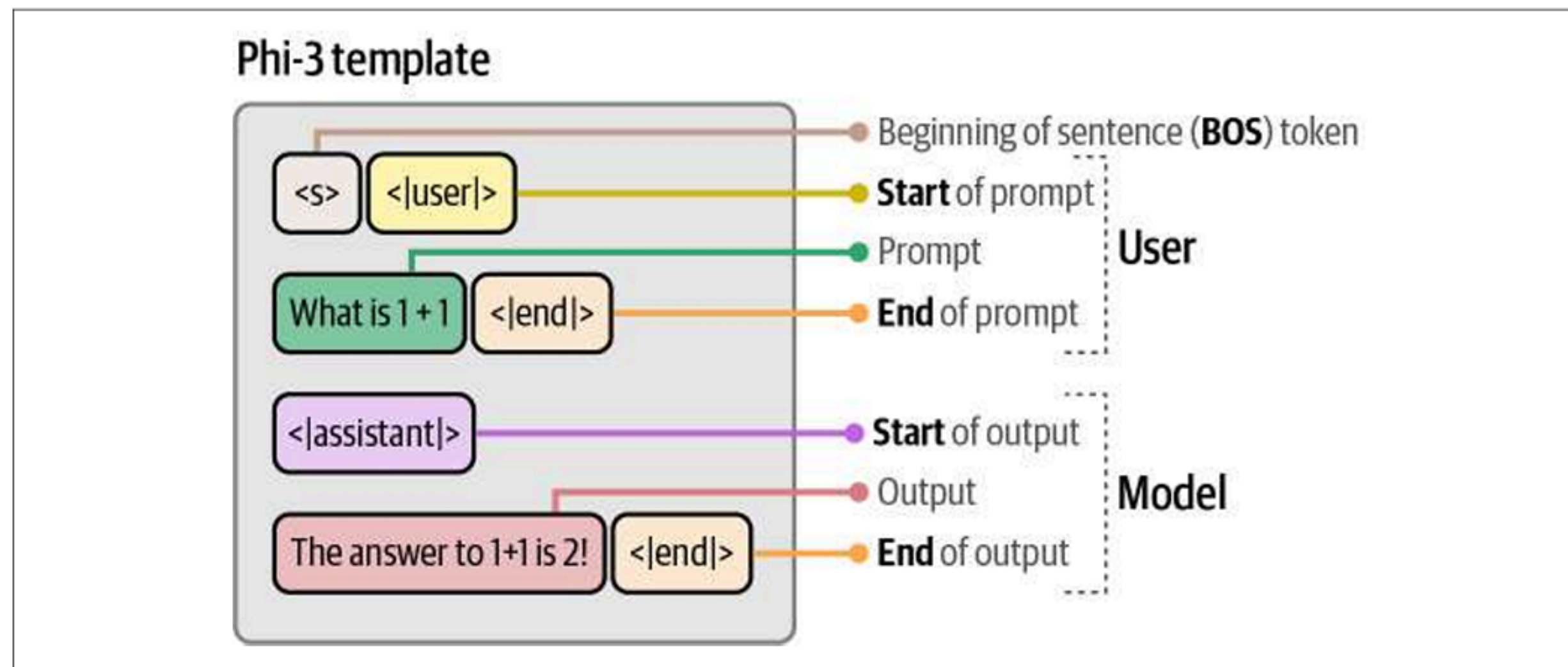


Figure 7-5. The prompt template Phi-3 expects.

A Chain with Multiple Prompts



- In our previous example, we created a single chain consisting of a prompt template and an LLM. Since our example was quite straightforward, the LLM had no issues dealing with the prompt. However, some applications are more involved and require lengthy or complex prompts to generate a response that captures those intricate details.
- Instead, we could break this complex prompt into smaller subtasks that can be run sequentially. This would require multiple calls to the LLM but with smaller prompts and intermediate outputs as shown in Figure 7-7.

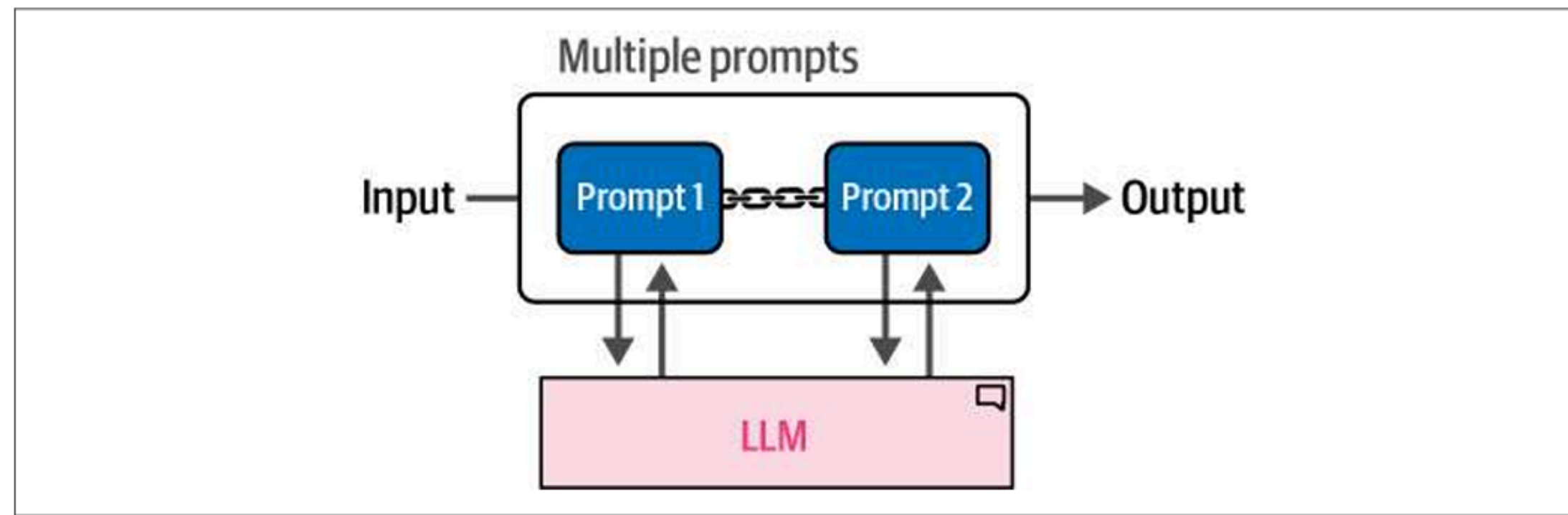


Figure 7-7. With sequential chains, the output of a prompt is used as the input for the next prompt.



- This process of using multiple prompts is an extension of our previous example. Instead of using a single chain, we link chains where each link deals with a specific subtask.
- For instance, consider the process of generating a story. We could ask the LLM to generate a story along with complex details like the title, a summary, a description of the characters, etc. Instead of trying to put all of that information into a single prompt, we could dissect this prompt into manageable smaller tasks instead.
- Let's illustrate with an example. Assume that we want to generate a story that has three components:
 - A title
 - A description of the main character
 - A summary of the story
 -
- Instead of generating everything in one go, we create a chain that only requires a single input by the user and then sequentially generates the three components. This process is illustrated in Figure 7-8.

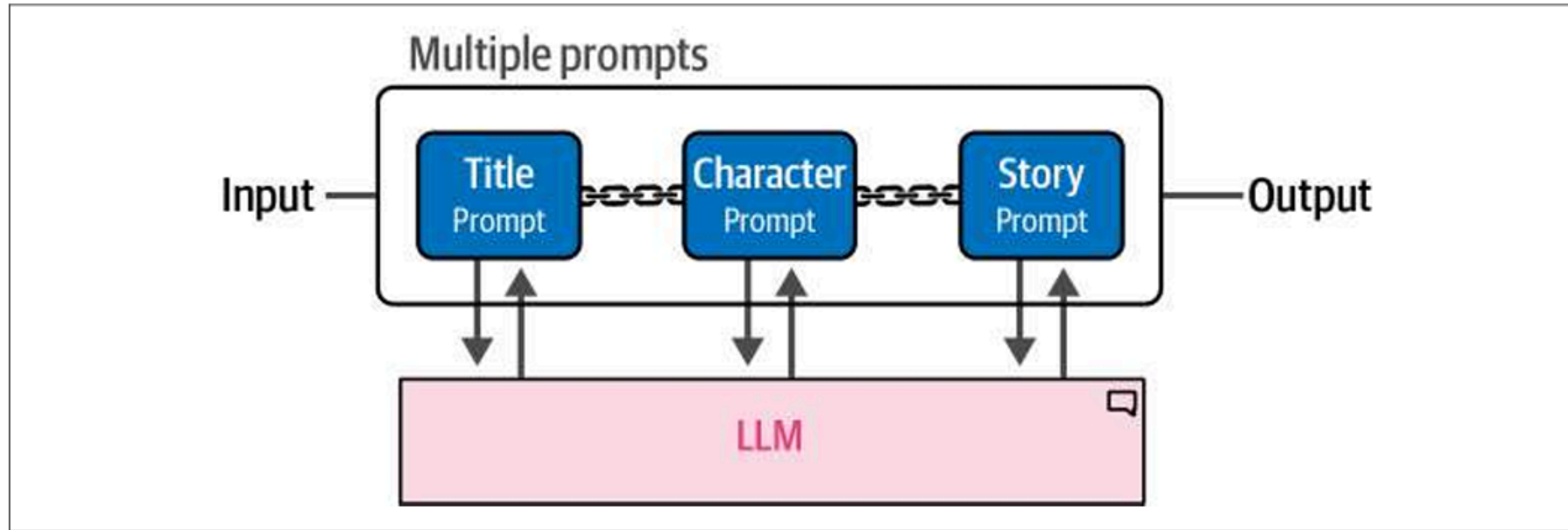


Figure 7-8. The output of the title prompt is used as the input of the character prompt. To generate the story, the output of all previous prompts is used.

Memory: Helping LLMs to Remember Conversations



- When we are using LLMs out of the box, they will not remember what was being said in a conversation. You can share your name in one prompt but it will have forgotten it by the next prompt.
- Let's illustrate this phenomenon with an example using the basic_chain we created before. First, we tell the LLM our name:

```
# Let's give the LLM our name  
basic_chain.invoke({"input_prompt": "Hi! My name is Maarten. What is 1 + 1?"})
```

Hello Maarten! The answer to 1 + 1 is 2.

Next, we ask it to reproduce the name we have given it:

```
# Next, we ask the LLM to reproduce the name  
basic_chain.invoke({"input_prompt": "What is my name?"})
```

I'm sorry, but as a language model, I don't have the ability to know personal information about individuals. You can provide the name you'd like to know more about, and I can help you with information or general inquiries related to it.

- Unfortunately, the LLM does not know the name we gave it. The reason for this forgetful behavior is that these models are stateless—they have no memory of any previous conversation!
- As illustrated in Figure 7-9, conversing with an LLM that does not have any memory is not the greatest experience.
- To make these models stateful, we can add specific types of memory to the chain that we created earlier. In this section, we will go through two common methods for helping LLMs to remember conversations:

- Conversation buffer
- Conversation summary

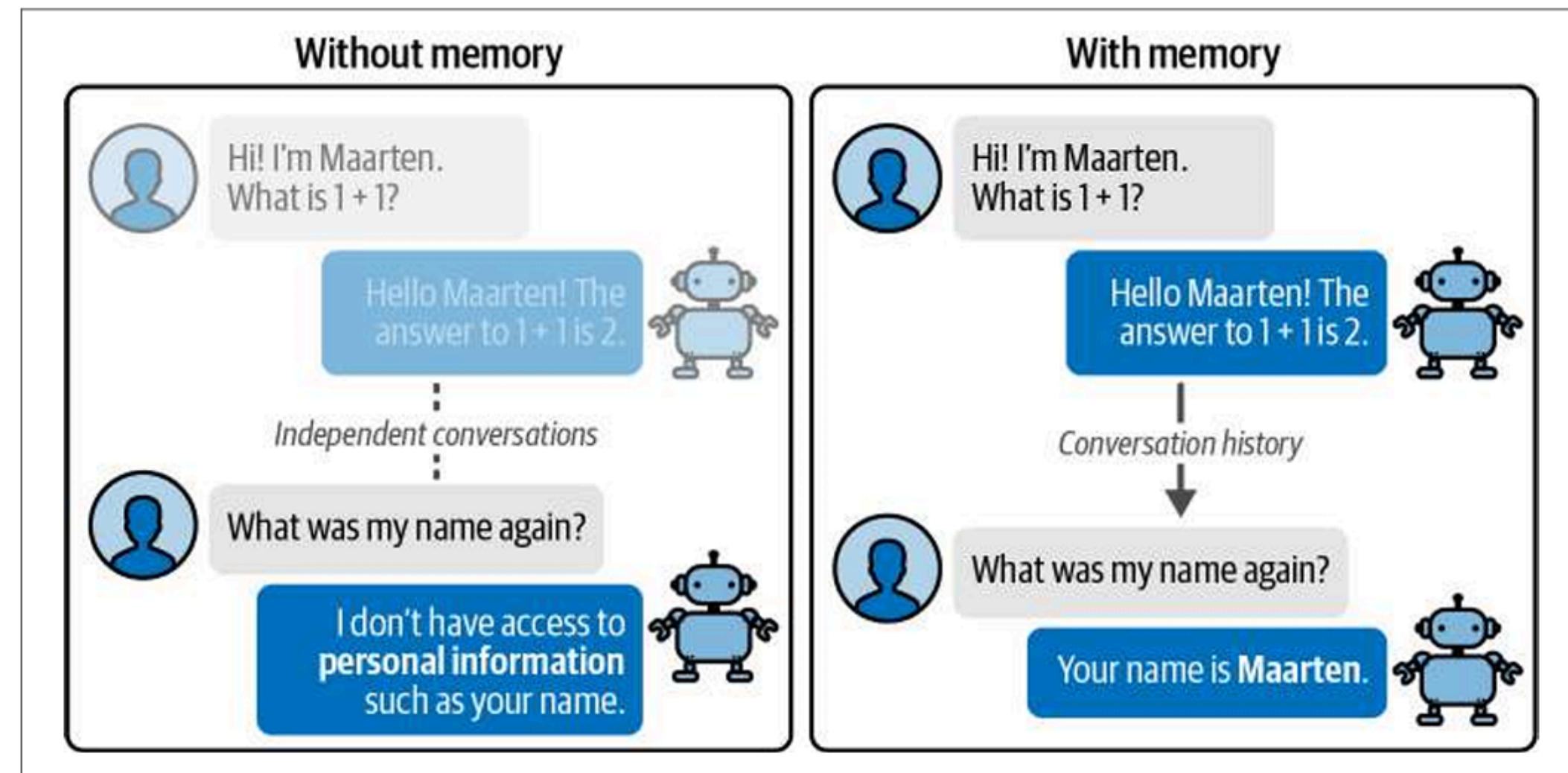


Figure 7-9. An example of a conversation between an LLM with memory and without memory.

Conversation Buffer



- One of the most intuitive forms of giving LLMs memory is simply reminding them exactly what has happened in the past. As illustrated in Figure 7-10, we can achieve this by copying the full conversation history and pasting that into our prompt.

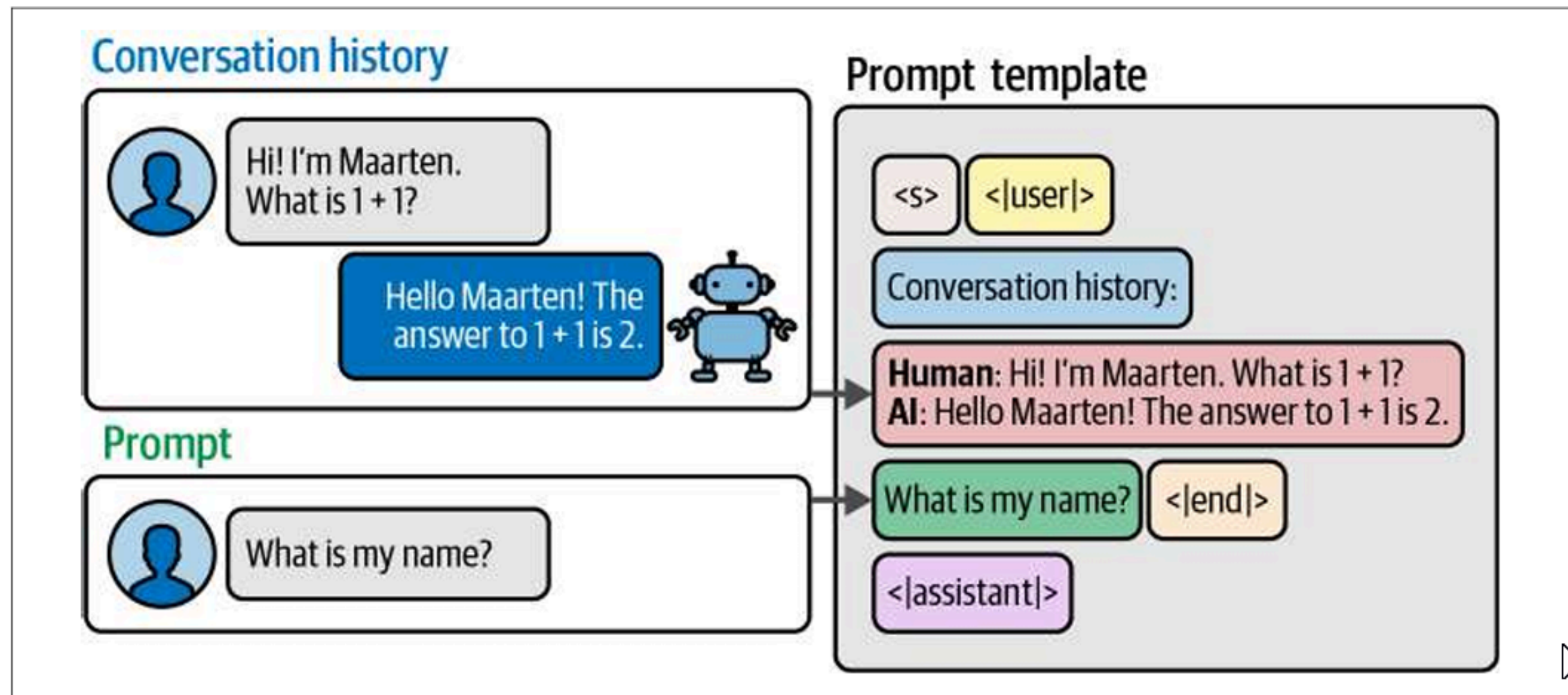


Figure 7-10. We can remind an LLM of what previously happened by simply appending the entire conversation history to the input prompt.



Windowed Conversation Buffer

- In our previous example, we essentially created a chatbot. You could talk to it and it remembers the conversation you had thus far. However, as the size of the conversation grows, so does the size of the input prompt until it exceeds the token limit. One method of minimizing the context window is to use the last k conversations instead of maintaining the full chat history. In LangChain, we can use Conversation BufferWindowMemory to decide how many conversations are passed to the input prompt

```
{'input_prompt': 'What is 3 + 3?',  
'chat_history': "Human: Hi! My name is Maarten and I am 33 years old. What is  
1 + 1?\nAI: Hello Maarten! It's nice to meet you. Regarding your question, 1 +  
1 equals 2. If you have any other questions or need further assistance, feel  
free to ask!\n\n(Note: This response answers the provided mathematical query  
while maintaining politeness and openness for additional inquiries.)",  
'text': " Hello Maarten! It's nice to meet you as well. Regarding your new  
question, 3 + 3 equals 6. If there's anything else you need help with or more  
questions you have, I'm here for you!"}
```

The interaction we had thus far is shown in "chat_history". Note that under the hood, LangChain saves it as an interaction between you (indicated with Human) and the LLM (indicated with AI).

Next, we can check whether the model indeed knows the name we gave it:

```
# Check whether it knows the name we gave it  
llm_chain.invoke({"input_prompt": "What is my name?"})
```

```
{'input_prompt': 'What is my name?',  
'chat_history': "Human: Hi! My name is Maarten and I am 33 years old. What is  
1 + 1?\nAI: Hello Maarten! It's nice to meet you. Regarding your question, 1 +  
1 equals 2. If you have any other questions or need further assistance, feel  
free to ask!\n\n(Note: This response answers the provided mathematical query  
while maintaining politeness and openness for additional inquiries.)\nHuman:  
What is 3 + 3?\nAI: Hello Maarten! It's nice to meet you as well. Regarding  
your new question, 3 + 3 equals 6. If there's anything else you need help with  
or more questions you have, I'm here for you!",  
'text': " Your name is Maarten, as mentioned at the beginning of our conversa-  
tion. Is there anything else you would like to know or discuss?"}
```

Conversation Summary



- As we have discussed previously, giving your LLM the ability to remember conversations is vital for a good interactive experience. However, when using Conversation BufferMemory, the conversation starts to increase in size and will slowly approach your token limit. Although ConversationBufferWindowMemory resolves the issue of token limits to an extent, only the last k conversations are retained.
- Although a solution would be to use an LLM with a larger context window, these tokens still need to be processed before generation tokens, which can increase compute time. Instead, let's look toward a more sophisticated technique, ConversationSummaryMemory. As the name implies, this technique summarizes an entire conversation history to distill it into the main points.
- This summarization process is enabled by another LLM that is given the conversation history as input and asked to create a concise summary. A nice advantage of using an external LLM is that we are not confined to using the same LLM during conversation. The summarization process is illustrated in Figure 7-12.

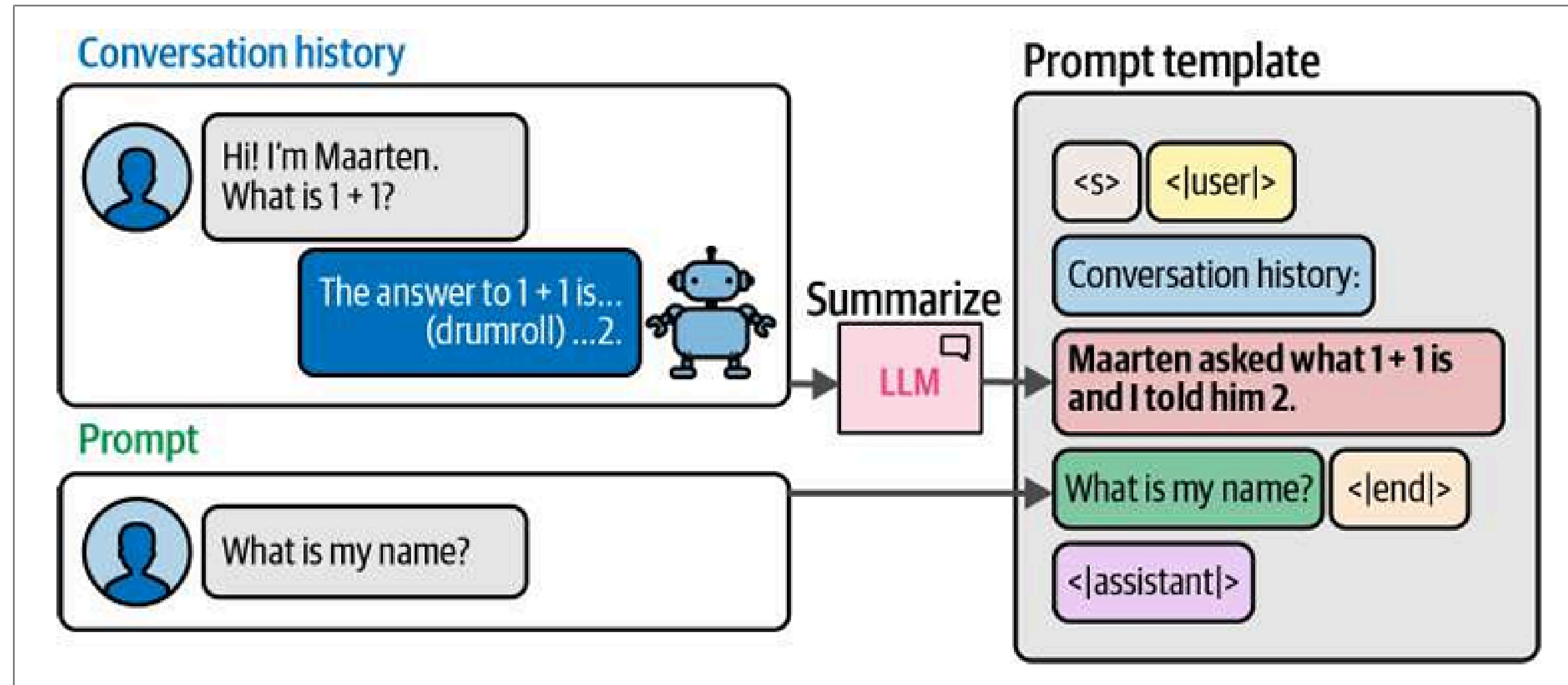


Figure 7-12. Instead of passing the conversation history directly to the prompt, we use another LLM to summarize it first.



Agents: Creating a System of LLMs

- Thus far, we have created systems that follow a user-defined set of steps to take. One of the most promising concepts in LLMs is their ability to determine the actions they can take. This idea is often called agents, systems that leverage a language model to determine which actions they should take and in what order.
- Agents can make use of everything we have seen thus far, such as model I/O, chains, and memory, and extend it further with two vital components:
- ***Tools that the agent can use to do things it could not do itself***
- ***The agent type, which plans the actions to take or tools to use***
- Unlike the chains we have seen thus far, agents are able to show more advanced behavior like creating and self-correcting a roadmap to achieve a goal. They can interact with the real world through the use of tools. As a result, these agents can perform a variety of tasks that go beyond what an LLM is capable of in isolation.
- For example, LLMs are notoriously bad at mathematical problems and often fail at solving simple math-based tasks but they could do much more if we provide access to a calculator. As illustrated in Figure 7-14, the underlying idea of agents is that they utilize LLMs not only to understand our query but also to decide which tool to use and when.

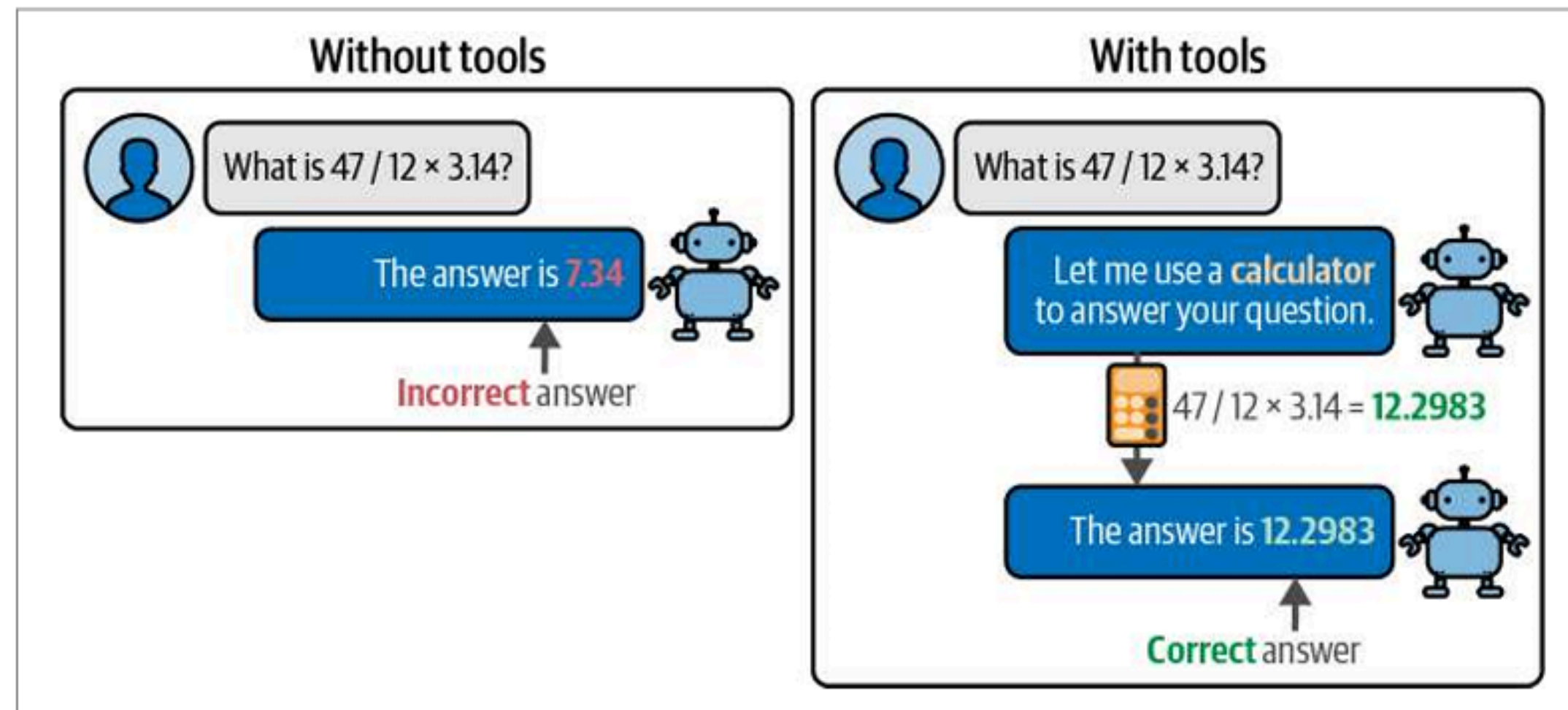


Figure 7-14. Giving LLMs the ability to choose which tools they use for a particular problem results in more complex and accurate behavior.

- In this example, we would expect the LLM to use the calculator when it faces a mathematical task. Now imagine we extend this with dozens of other tools, like a search engine or a weather API. Suddenly, the capabilities of LLMs increase significantly
- In other words, agents that make use of LLMs can be powerful general problem solvers. Although the tools they use are important, the driving force of many agent-based systems is the use of a framework called Reasoning and Acting (ReAct1).

COURSE 1

LARGE LANGUAGE MODELS (LLMs) & AGENTIC AI

